

Министерство науки и высшего образования Российской
Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
**«Национальный исследовательский университет
ИТМО»**

Курсовая работа
по дисциплине «Объектно-ориентированное
программирование»

Учет внутриофисных расходов

Выполнил:
студент группы _____

Проверил:

Санкт-Петербург
2024

Оглавление

1	Введение	2
2	Задание 1: Реализация классов сущностей	3
2.1	Пояснение к выполнению Задания 1	3
2.1.1	Модуль <code>general.py</code>	3
2.1.2	Модуль <code>department.py</code>	3
2.1.3	Модуль <code>expensetype.py</code>	4
2.1.4	Модуль <code>expense.py</code>	4
2.2	UML-диаграмма классов сущностей	5
3	Задание 2: Реализация системы хранения данных	6
3.1	Пояснение к выполнению Задания 2	6
3.1.1	Модуль <code>generallist.py</code>	6
3.1.2	Модуль <code>departmentlist.py</code>	7
3.1.3	Модуль <code>expensetypelist.py</code>	7
3.1.4	Модуль <code>expenselist.py</code>	8
3.1.5	Модуль <code>data.py</code>	8
3.1.6	Модуль <code>dataxml.py</code>	9
3.1.7	Модуль <code>accounting.py</code>	10
3.1.8	Модуль <code>rowcode.py</code>	11
3.1.9	Пример XML-файла данных	12
3.2	UML-диаграмма слоя данных	12
4	Задание 3: Реализация графического интерфейса PyQt5	13
4.1	Пояснение к выполнению Задания 3	13
4.1.1	Модуль <code>libwidget.py</code>	13
4.1.2	Модуль <code>dbcombobox.py</code>	14
4.1.3	Модуль <code>dbtablewidget.py</code>	14
4.1.4	Модуль <code>editform.py</code>	15
4.1.5	Модуль <code>page.py</code>	16
4.1.6	Таблицы для сущностей	17
4.1.7	Комбо-боксы для выбора связанных объектов	18
4.1.8	Формы редактирования	19
4.1.9	Страницы	21

4.1.10	Модуль <code>tabwidget.py</code>	22
4.1.11	Модуль <code>mainwindow.py</code>	23
4.1.12	Точка входа: <code>main.pyw</code>	24
4.2	UML-диаграмма классов GUI	24
4.3	Результаты работы приложения	24
4.3.1	Вкладка «Отделы»	25
4.3.2	Вкладка «Виды расходов»	26
4.3.3	Вкладка «Расходы»	27
4.3.4	Загрузка данных из XML	27
4.3.5	Сохранение данных в XML	28

5	Заключение	29
----------	-------------------	-----------

Глава 1

Введение

Целью данной курсовой работы является разработка приложения на языке Python с графическим интерфейсом PyQt5 для автоматизации учёта внутри-офисных расходов частной фирмы. Приложение реализует объектно-ориентированный подход к моделированию предметной области и поддерживает сохранение/загрузку данных в формате XML.

Предметная область включает три сущности:

- **Отделы** (Название, Количество сотрудников)
- **Виды расходов** (Название, Описание, Предельная норма)
- **Расходы** (Вид расхода, Отдел, Сумма, Дата)

Приложение построено по многослойной архитектуре: слой данных (классы-сущности и списки), слой хранения (чтение/запись XML), слой графического интерфейса (таблицы, формы редактирования, страницы с вкладками).

Глава 2

Задание 1: Реализация классов сущностей

2.1 Пояснение к выполнению Задания 1

Согласно предоставленным источникам, первым этапом разработки является создание базовых классов, моделирующих сущности предметной области. В основе иерархии лежит класс `general`, который обрабатывает общие атрибуты `code` и `name`, используя принцип обобщения. Все остальные сущности наследуются от него, добавляя специфические атрибуты через методы доступа (`set/get`), обеспечивающие инкапсуляцию данных.

2.1.1 Модуль `general.py`

Этот базовый класс обрабатывает общие атрибуты `code` и `name`, используя принцип обобщения.

Листинг 2.1: Модуль `general.py` — базовый класс сущностей

```
1 class general:
2     def __init__(self, code=0, name=''):
3         self.setCode(code)
4         self.setName(name)
5     def setCode(self, value): self.__code=value
6     def setName(self, value): self.__name=value
7     def getCode(self): return self.__code
8     def getName(self): return self.__name
```

2.1.2 Модуль `department.py`

Класс для сущности «Отдел» (`department`), наследуемый от `general`. Добавляет атрибут `employees` (количество сотрудников).

Листинг 2.2: Модуль `department.py` — сущность «Отдел»

```
1 from general import general
```

```

2
3 class department(general):
4     def __init__(self, code=0, name='', employees=0):
5         general.__init__(self, code, name)
6         self.setEmployees(employees)
7     def setEmployees(self, value): self.__employees=value
8     def getEmployees(self): return self.__employees

```

2.1.3 Модуль expensetype.py

Класс для сущности «Вид расхода» (expensetype), включающий описание и предельную норму.

Листинг 2.3: Модуль expensetype.py — сущность «Вид расхода»

```

1 from general import general
2
3 class expensetype(general):
4     def __init__(self, code=0, name='', description='', limit=0):
5         general.__init__(self, code, name)
6         self.setDescription(description)
7         self.setLimit(limit)
8     def setDescription(self, value): self.__description=value
9     def setLimit(self, value): self.__limit=value
10    def getDescription(self): return self.__description
11    def getLimit(self): return self.__limit

```

2.1.4 Модуль expense.py

Класс «Расход» (expense), который связывает отдел и вид расхода через агрегацию. Содержит поля amount (сумма) и date (дата), а также вспомогательные методы для получения названий связанных объектов.

Листинг 2.4: Модуль expense.py — сущность «Расход»

```

1 from general import general
2 from department import department
3 from expensetype import expensetype
4
5 class expense(general):
6     def __init__(self, code=0, name='', expensetype=None, department=None, amount
7         =0, date=''):
8         general.__init__(self, code, name)
9         self.setExpenseType(expensetype)
10        self.setDepartment(department)
11        self.setAmount(amount)
12        self.setDate(date)
13    def setExpenseType(self, value): self.__expensetype=value
14    def setDepartment(self, value): self.__department=value
15    def setAmount(self, value): self.__amount=value
16    def setDate(self, value): self.__date=value
17    def getExpenseType(self): return self.__expensetype
18    def getExpenseTypeCode(self):
19        if self.getExpenseType(): return self.getExpenseType().getCode()
20        else: return 0

```

```
20 def getExpenseTypeName(self):
21     if self.getExpenseType():return self.getExpenseType().getName()
22     else:return ''
23 def getDepartment(self):return self.__department
24 def getDepartmentCode(self):
25     if self.getDepartment():return self.getDepartment().getCode()
26     else:return 0
27 def getDepartmentName(self):
28     if self.getDepartment():return self.getDepartment().getName()
29     else:return ''
30 def getAmount(self):return self.__amount
31 def getDate(self):return self.__date
```

2.2 UML-диаграмма классов сущностей

На рисунке 2.1 представлена диаграмма классов, отражающая отношения наследования и агрегации между сущностями. Класс **expense** агрегирует как **department**, так и **expensetype**, в то время как все три наследуют от **general**.

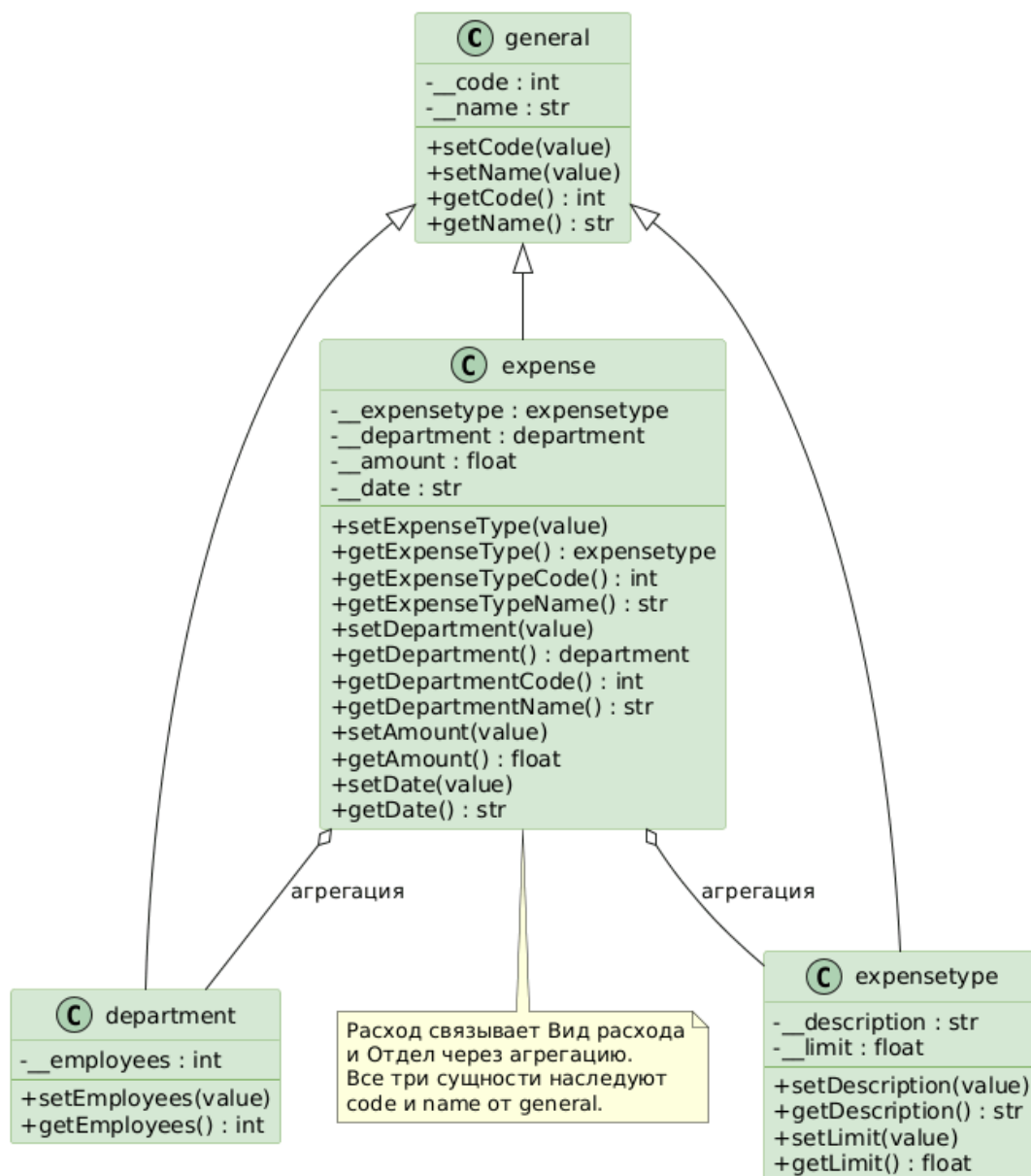


Рис. 2.1: UML-диаграмма классов сущностей

Глава 3

Задание 2: Реализация системы хранения данных

3.1 Пояснение к выполнению Задания 2

Согласно источникам, реализация системы хранения данных строится на следующих принципах:

1. **Специализированные списки:** Для каждой сущности создаются классы-списки (наследники `generalList`), которые обеспечивают контроль типов и фабричные методы для создания объектов.
2. **Центральный управляющий класс:** Создаётся класс `accounting`, который объединяет все списки и координирует работу с ними.
3. **Иерархия классов данных:** Базовый класс `data` определяет общие атрибуты для работы с файлами, а его наследник `dataxml` реализует конкретную логику чтения и записи XML через библиотеку `xml.dom.minidom`.

3.1.1 Модуль `generallist.py`

Базовый класс списка, предоставляющий общие методы: поиск по коду, получение нового кода, добавление/удаление элементов.

Листинг 3.1: Модуль `generallist.py` — базовый класс списка

```
1 from general import general
2
3 class generalList:
4     def __init__(self): self.__list=[]
5     def clear(self): self.__list=[]
6     def findByCode(self, code):
7         for l in self.__list:
8             if l.getCode()==code:
9                 return l
10    def getNewCode(self):
11        l=self.getCodes()
```

```

12         if l:return max(l)+1
13         else:return 1
14     def getCodes(self):return [s.getCode() for s in self.__list]
15     def getItems(self):return [s for s in self.__list]
16     def appendItem(self,value):
17         self.__list.append(value)
18     def removeItem(self,value):
19         if isinstance(value,general):self.__list.remove(value)
20         if isinstance(value,int):self.__list.remove(self.findByCode(value))

```

3.1.2 Модуль departmentlist.py

Класс для управления коллекцией объектов department. Переопределяет appendItem с проверкой типа и добавляет фабричные методы createItem и newItem.

Листинг 3.2: Модуль departmentlist.py — список отделов

```

1 from department import department
2 from generallist import generalList
3
4 class departmentList(generalList):
5     def appendItem(self,value):
6         if isinstance(value,department):generalList.appendItem(self,value)
7     def createItem(self,code,name,employees=0):
8         if code in self.getCodes():print('Отдел с кодом %s уже существует'%(
9             code,))
10        else:
11            d=department(code,name,employees)
12            self.appendItem(d)
13            return d
14    def newItem(self,name,employees=0):
15        d=department(self.getNewCode(),name,employees)
16        self.appendItem(d)
17        return d

```

3.1.3 Модуль expensetypelist.py

Класс для управления коллекцией объектов expensetype.

Листинг 3.3: Модуль expensetypelist.py — список видов расходов

```

1 from expensetype import expensetype
2 from generallist import generalList
3
4 class expensetypeList(generalList):
5     def appendItem(self,value):
6         if isinstance(value,expensetype):generalList.appendItem(self,value)
7     def createItem(self,code,name,description='',limit=0):
8         if code in self.getCodes():print('Вид расхода с кодом %s уже существ
9             yet'%(code,))
10        else:
11            e=expensetype(code,name,description,limit)
12            self.appendItem(e)
13            return e
14    def newItem(self,name,description='',limit=0):
15        e=expensetype(self.getNewCode(),name,description,limit)

```

```

15         self.appendItem(e)
16         return e

```

3.1.4 Модуль `expenselist.py`

Класс для управления коллекцией объектов `expense`.

Листинг 3.4: Модуль `expenselist.py` — список расходов

```

1  from expense import expense
2  from generalist import generalList
3
4  class expenseList(generalList):
5      def appendItem(self, value):
6          if isinstance(value, expense): generalList.appendItem(self, value)
7      def createItem(self, code, name='', expensetype=None, department=None, amount
8          =0, date=''):
9          if code in self.getCodes(): print('Расход с кодом %s уже существует'
10              %(code,))
11          else:
12              e=expense(code, name, expensetype, department, amount, date)
13              self.appendItem(e)
14              return e
15      def newItem(self, name='', expensetype=None, department=None, amount=0, date=
16          ''):
17          e=expense(self.getNewCode(), name, expensetype, department, amount, date)
18          self.appendItem(e)
19          return e

```

3.1.5 Модуль `data.py`

Базовый класс для операций чтения/записи данных. Содержит ссылку на управляющий класс (`lib`) и имена входного/выходного файлов. Методы `read()` и `write()` переопределяются в наследниках.

Листинг 3.5: Модуль `data.py` — базовый класс данных

```

1  class data:
2      def __init__(self, lib=None, inp='', out=''):
3          self.setLib(lib)
4          self.setInp(inp)
5          self.setOut(out)
6
7      def setLib(self, value): self.__lib=value
8      def setInp(self, value): self.__inp=value
9      def setOut(self, value): self.__out=value
10
11     def getLib(self): return self.__lib
12     def getInp(self): return self.__inp
13     def getOut(self): return self.__out
14
15     def readFile(self, lib=None, filename=''):
16         if lib: self.setLib(lib)
17         if filename: self.setInp(filename)
18         self.read()
19     def writeFile(self, lib=None, filename=''):

```

```

20         if lib:self.setLib(lib)
21         if filename:self.setOut(filename)
22         self.write()
23
24     def read(self):pass
25     def write(self):pass

```

3.1.6 Модуль dataxml.py

Реализация чтения и записи данных в формате XML. При чтении парсит XML-документ и восстанавливает связи между объектами по кодам. При записи формирует структуру XML с вложенными атрибутами.

Листинг 3.6: Модуль dataxml.py — работа с XML

```

1  import os,xml.dom.minidom
2  from data import data
3
4  class dataxml(data):
5      def read(self):
6          dom=xml.dom.minidom.parse(self.getInp())
7          dom.normalize()
8          for node in dom.childNodes[0].childNodes:
9              if (node.nodeType==node.ELEMENT_NODE)and(node.nodeName=='
                department'):
10                 code,name,employees=0,"",0
11                 for t in node.attributes.items():
12                     if t[0]=="code":code=int(t[1])
13                     if t[0]=="name":name=t[1]
14                     if t[0]=="employees":employees=int(t[1])
15                 self.getLib().createDepartment(code,name,employees)
16             if (node.nodeType==node.ELEMENT_NODE)and(node.nodeName=='
                expensetype'):
17                 code,name,description,limit=0,"","",0
18                 for t in node.attributes.items():
19                     if t[0]=="code":code=int(t[1])
20                     if t[0]=="name":name=t[1]
21                     if t[0]=="description":description=t[1]
22                     if t[0]=="limit":limit=float(t[1])
23                 self.getLib().createExpenseType(code,name,description,limit)
24             if (node.nodeType==node.ELEMENT_NODE)and(node.nodeName=='expense
                '):
25                 code,name,expensetype,department,amount,date=0','','None,None
26                 ,0,''
27                 for t in node.attributes.items():
28                     if t[0]=="code":code=int(t[1])
29                     if t[0]=="name":name=t[1]
30                     if t[0]=="amount":amount=float(t[1])
31                     if t[0]=="date":date=t[1]
32                     if t[0]=="expensetype":expensetype=self.getLib().
33                     getExpenseType(int(t[1]))
34                     if t[0]=="department":department=self.getLib().
35                     getDepartment(int(t[1]))
36                 self.getLib().createExpense(code,name,expensetype,department
                ,amount,date)
37
38     def write(self):
39         dom=xml.dom.minidom.Document()
40         root=dom.createElement("accounting")

```

```

37     dom.appendChild(root)
38     for d in self.getLib().getDepartmentList():
39         dep=dom.createElement("department")
40         dep.setAttribute('code',str(d.getCode()))
41         dep.setAttribute('name',d.getName())
42         dep.setAttribute('employees',str(d.getEmployees()))
43         root.appendChild(dep)
44     for t in self.getLib().getExpenseTypeList():
45         et=dom.createElement("expensetype")
46         et.setAttribute('code',str(t.getCode()))
47         et.setAttribute('name',t.getName())
48         et.setAttribute('description',t.getDescription())
49         et.setAttribute('limit',str(t.getLimit()))
50         root.appendChild(et)
51     for e in self.getLib().getExpenseList():
52         exp=dom.createElement("expense")
53         exp.setAttribute('code',str(e.getCode()))
54         exp.setAttribute('name',e.getName())
55         exp.setAttribute('amount',str(e.getAmount()))
56         exp.setAttribute('date',e.getDate())
57         exp.setAttribute('expensetype',str(e.getExpenseTypeCode()))
58         exp.setAttribute('department',str(e.getDepartmentCode()))
59         root.appendChild(exp)
60     f = open(self.getOut(),"w",encoding='utf-8')
61     f.write(dom.toprettyxml())

```

3.1.7 Модуль accounting.py

Центральный управляющий класс (аналог `library` из источников), объединяющий все списки сущностей. Предоставляет унифицированный интерфейс для создания, получения и удаления объектов всех трёх типов. При удалении отдела или вида расхода обновляет связанные расходы (устанавливает ссылку в `None`).

Листинг 3.7: Модуль `accounting.py` — управляющий класс

```

1  from departmentlist import departmentList
2  from expensetypelist import expensetypeList
3  from expenselist import expenseList
4
5  class accounting:
6      def __init__(self):
7          self.__departmentList=departmentList()
8          self.__expensetypeList=expensetypeList()
9          self.__expenseList=expenseList()
10     def clear(self):
11         self.__expenseList.clear()
12         self.__departmentList.clear()
13         self.__expensetypeList.clear()
14     def createDepartment(self,code,name,employees=0):return self.__departmentList.createItem(code,name,employees)
15     def newDepartment(self,name,employees=0):return self.__departmentList.newItem(name,employees)
16     def removeDepartment(self,code):
17         self.__departmentList.removeItem(code)
18         for e in self.__expenseList.getItems():
19             if e.getDepartmentCode()==code:e.setDepartment(None)

```

```

20 def getDepartment(self, code): return self.__departmentList.findByCode(
    code)
21 def getDepartmentList(self): return self.__departmentList.getItems()
22 def getDepartmentCodes(self): return self.__departmentList.getCodes()
23 def getDepartmentNewCode(self): return self.__departmentList.getNewCode()
24
25 def createExpenseType(self, code, name, description='', limit=0): return self
    .__expensetypeList.createItem(code, name, description, limit)
26 def newExpenseType(self, name, description='', limit=0): return self.
    __expensetypeList.newItem(name, description, limit)
27 def removeExpenseType(self, code):
28     self.__expensetypeList.removeItem(code)
29     for e in self.__expenseList.getItems():
30         if e.getExpenseTypeCode() == code: e.setExpenseType(None)
31 def getExpenseType(self, code): return self.__expensetypeList.findByCode(
    code)
32 def getExpenseTypeList(self): return self.__expensetypeList.getItems()
33 def getExpenseTypeCodes(self): return self.__expensetypeList.getCodes()
34 def getExpenseTypeNewCode(self): return self.__expensetypeList.getNewCode
    ()
35
36 def createExpense(self, code, name='', expensetype=None, department=None,
    amount=0, date=''):
37     return self.__expenseList.createItem(code, name, expensetype,
        department, amount, date)
38 def newExpense(self, name='', expensetype=None, department=None, amount=0,
    date=''):
39     return self.__expenseList.newItem(name, expensetype, department, amount
        , date)
40 def removeExpense(self, code): self.__expenseList.removeItem(code)
41 def getExpense(self, code): return self.__expenseList.findByCode(code)
42 def getExpenseList(self): return self.__expenseList.getItems()
43 def getExpenseCodes(self): return self.__expenseList.getCodes()
44 def getExpenseNewCode(self): return self.__expenseList.getNewCode()

```

3.1.8 Модуль rowcode.py

Вспомогательный класс для отображения соответствия между строками таблицы и кодами объектов. Используется в виджетах графического интерфейса.

Листинг 3.8: Модуль rowcode.py — отображение строки-кода

```

1 class rowCode:
2     def __init__(self): self.__list=[]
3     def clear(self): self.__list=[]
4     def appendRowCode(self, row, code): self.__list.append([row, code])
5     def updateRow(self, row):
6         for t in self.__list:
7             if t[0]>row: t[0]-=1
8     def removeRow(self, row):
9         for t in self.__list:
10            if t[0]==row: self.__list.remove(t)
11            self.updateRow(row)
12     def removeCode(self, code):
13         for t in self.__list:
14            if t[1]==code: self.__list.remove(t)
15     def getCode(self, row):

```

```

16         for t in self.__list:
17             if t[0]==row:return t[1]
18     def getCodes(self):return [t[1] for t in self.__list]
19     def getRow(self,code):
20         for t in self.__list:
21             if t[1]==code:return t[0]

```

3.1.9 Пример XML-файла данных

Листинг 3.9: Файл data.xml — пример данных в формате XML

```

1 <accounting>
2     <department code="1" name="Бухгалтерия" employees="5"/>
3     <department code="2" name="Отдел продаж" employees="12"/>
4     <department code="3" name="ИТ-отдел" employees="8"/>
5
6     <expensetype code="1" name="Канцелярские товары" description="Бумага, ру
7         чки, карандаши" limit="5000"/>
8     <expensetype code="2" name="Расходные материалы" description="Картриджи,
9         тонер" limit="10000"/>
10    <expensetype code="3" name="Транспортные расходы" description="Такси, об
11        щественный транспорт" limit="15000"/>
12
13    <expense code="1" name="" amount="1200" date="2024-01-15" expensetype="1
14        " department="1"/>
15    <expense code="2" name="" amount="3500" date="2024-01-20" expensetype="2
16        " department="3"/>
17    <expense code="3" name="" amount="800" date="2024-02-05" expensetype="3"
18        department="2"/>
19    <expense code="4" name="" amount="2100" date="2024-02-10" expensetype="1
20        " department="2"/>
21 </accounting>

```

3.2 UML-диаграмма слоя данных

На рисунке 3.1 представлена диаграмма классов слоя данных, отражающая отношения между управляющим классом `accounting`, специализированными списками, базовым классом `data` и его наследником `dataxml`.

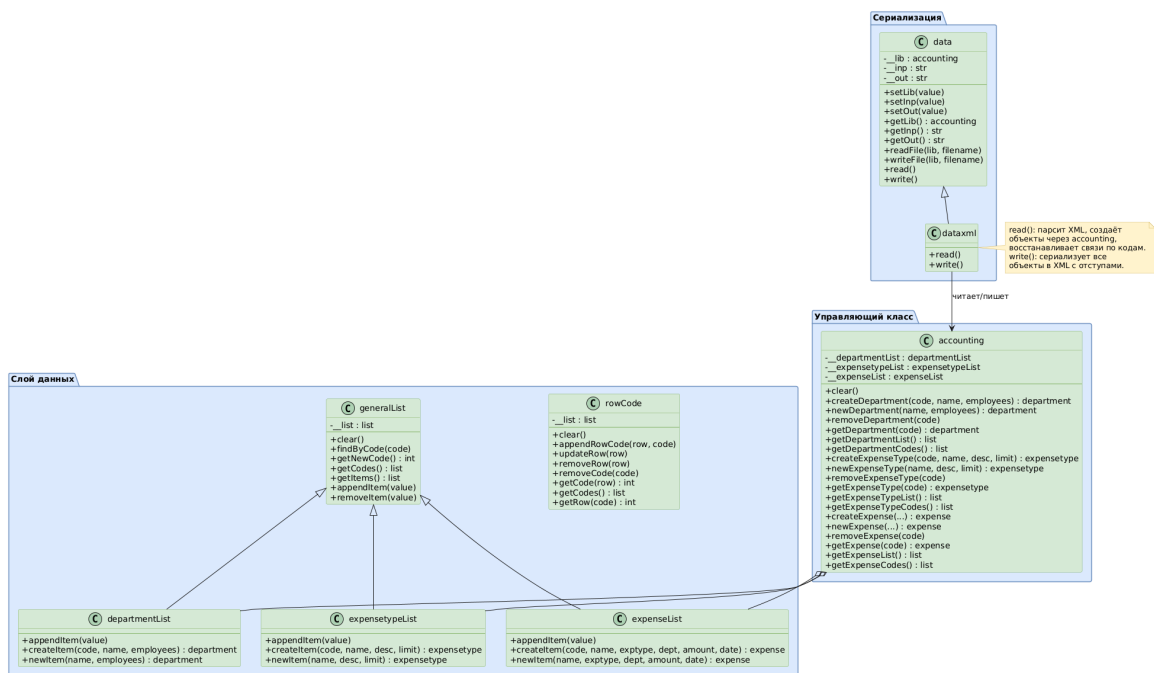


Рис. 3.1: UML-диаграмма классов слоя данных

Глава 4

Задание 3: Реализация графического интерфейса PyQt5

4.1 Пояснение к выполнению Задания 3

В соответствии с источниками, третий этап разработки заключается в создании полноценного графического интерфейса на базе библиотеки PyQt5. Архитектура GUI повторяет шаблон, продемонстрированный в учебных примерах `example4` и `example5`:

1. **Базовые виджеты:** `libWidget` — примесь для доступа к данным; `dbTableWidget` — таблица с привязкой строк к кодам; `dbComboBox` — выпадающий список с привязкой индексов к кодам.
2. **Базовые формы:** `editForm` предоставляет сетку для размещения полей ввода и кнопок; `page` объединяет таблицу, форму редактирования и кнопки управления (добавить/изменить/удалить).
3. **Специализированные виджеты:** Для каждой сущности создаются таблица, форма редактирования и страница. Для расхода дополнительно создаются комбо-боксы (`departmentCombo`, `expensetypeCombo`) для выбора связанных объектов.
4. **Главное окно:** `tabWidget` объединяет страницы во вкладки; `mainWindow` добавляет меню (создать новый, открыть XML, сохранить XML).

4.1.1 Модуль `libwidget.py`

Примесь (`mixin`), предоставляющая доступ к управляющему объекту данных `library`.

Листинг 4.1: Модуль libwidget.py — примесь для доступа к данным

```
1 class libWidget:
2     def __init__(self, library=None):
3         self.__library=library
4     def getLibrary(self):return self.__library
```

4.1.2 Модуль dbcombobox.py

Базовый выпадающий список, использующий `rowCode` для отображения индексов на коды объектов. Поддерживает методы `getCurrentCode()`, `setCurrentRec()` и динамическое обновление через `update()`.

Листинг 4.2: Модуль dbcombobox.py — выпадающий список с кодами

```
1 from PyQt5.QtWidgets import QComboBox
2 from rowcode import rowCode
3 from libwidget import libWidget
4
5 class dbComboBox(QComboBox, libWidget):
6     def __init__(self, parent=None, library=None):
7         QComboBox.__init__(self, parent)
8         libWidget.__init__(self, library)
9         self.__rowCode=rowCode()
10        self.setSizeAdjustPolicy(self.AdjustToContents)
11    def clear(self):
12        self.__rowCode.clear()
13        QComboBox.clear(self)
14    def addItem(self, code, text):
15        self.__rowCode.appendRowCode(self.count(), code)
16        QComboBox.addItem(self, text)
17    def removeItem(self, index):
18        self.__rowCode.removeRow(index)
19        QComboBox.removeItem(self, index)
20    def getCurrentCode(self):return self.__rowCode.getCode(self.currentIndex())
21    def setCurrentCode(self, code):
22        if self.__rowCode.getRow(code):self.setCurrentIndex(self.__rowCode.getRow(code))
23    def setCurrentRec(self, value):
24        self.__currentRec=value
25        self.update()
26    def getCurrentRec(self):return self.__currentRec
27    def update(self):pass
```

4.1.3 Модуль dbtablewidget.py

Базовая таблица с автоматической привязкой строк к кодам объектов через `rowCode`. Метод `setData()` переопределяется в наследниках для заполнения таблицы конкретными данными.

Листинг 4.3: Модуль dbtablewidget.py — таблица с кодами

```
1 from PyQt5.QtWidgets import QTableWidget
2 from rowcode import rowCode
3 from libwidget import libWidget
```

```

4
5 class dbTableWidget(QTableWidget, libWidget):
6     def __init__(self, library, parent=None, header=[]):
7         QTableWidget.__init__(self)
8         libWidget.__init__(self, library)
9         self.__rowCode=rowCode()
10        self.setHeader(header)
11        self.update()
12    def setHeader(self, value):
13        self.setColumnCount(len(value))
14        self.setHorizontalHeaderLabels(value)
15    def clearContents(self):
16        self.__rowCode.clear()
17        QTableWidget.clearContents(self)
18    def getCodes(self):return self.__rowCode.getCodes()
19    def getCurrentCode(self):return self.__rowCode.getCode(self.currentRow()
20    )
21    def setCurrentCode(self, code):
22        if code:self.setCurrentCell(self.__rowCode.getRow(code),0)
23    def appendRowCode(self, row, code):self.__rowCode.appendRowCode(row, code)
24    def update(self, code=0):
25        self.clearContents()
26        self.setData()
27        self.resizeColumnsToContents()
28        self.resizeRowsToContents()
29        self.setCurrentCode(code)
30    def setData(self):pass

```

4.1.4 Модуль editform.py

Базовая форма редактирования с сеточным размещением виджетов. Предоставляет методы `addLabel()`, `addNewWidget()`, `addLeftLayout()` для построения интерфейса, а также шаблонные методы `newClick()`, `editClick()`, `delClick()`, переопределяемые в наследниках.

Листинг 4.4: Модуль `editform.py` — базовая форма редактирования

```

1 from PyQt5.QtWidgets import QWidget, QGridLayout, QVBoxLayout, QHBoxLayout,
2   QPushButton, QLabel
3
4 from libwidget import libWidget
5
6 class editForm(QWidget, libWidget):
7     def __init__(self, library=None, parent=None):
8         QWidget.__init__(self, parent=parent)
9         libWidget.__init__(self, library)
10        self.__grid=QGridLayout()
11        self.__vbox=QVBoxLayout()
12        self.__hbox=QHBoxLayout()
13        self.__vbox.addLayout(self.__grid)
14        self.__vbox.addStretch(1)
15        self.__hbox.addLayout(self.__vbox)
16        self.setLayout(self.__hbox)
17
18    def addLabel(self, text, x, y):self.__grid.addWidget(QLabel(text), x, y)
19    def addNewWidget(self, widget, x, y):self.__grid.addWidget(widget, x, y)
20    def addLeftLayout(self, layout):self.__hbox.insertLayout(0, layout)
21    def setCurrentCode(self, value):
22        self.__currentCode=value

```

```

20         self.update()
21     def getCurrentCode(self): return self.__currentCode
22     def decode(self, qstring): return str(qstring.toUtf8()).decode('utf-8')
23     def update(self): pass

```

4.1.5 Модуль page.py

Базовая страница, объединяющая таблицу (`setTable`), форму редактирования (`setForm`) и кнопки управления (добавить, изменить, удалить). Метод `setConnect()` связывает сигналы кнопок с соответствующими обработчиками.

Листинг 4.5: Модуль `page.py` — базовая страница с таблицей и формой

```

1 from PyQt5.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton
2 from libwidget import libWidget
3
4 class page(QWidget, libWidget):
5     def __init__(self, library, parent=None):
6         QWidget.__init__(self, parent=parent)
7         libWidget.__init__(self, library)
8         self.__vbox=QVBoxLayout()
9         self.__hbox=QHBoxLayout()
10        self.__buttonbox=QVBoxLayout()
11        self.__newButton=QPushButton("добавить")
12        self.__editButton=QPushButton("изменить")
13        self.__delButton=QPushButton("удалить")
14        self.__vbox.addLayout(self.__hbox)
15        self.__hbox.addLayout(self.__buttonbox)
16        self.__buttonbox.addWidget(self.__newButton)
17        self.__buttonbox.addWidget(self.__editButton)
18        self.__buttonbox.addWidget(self.__delButton)
19        self.__buttonbox.addStretch(1)
20        self.setLayout(self.__vbox)
21    def setTable(self, value):
22        self.__table=value
23        self.__vbox.insertWidget(0,self.__table)
24    def setForm(self, value):
25        self.__form=value
26        self.__hbox.insertWidget(0,self.__form)
27    def newClick(self):
28        self.__form.newClick()
29        self.__table.update(self.__form.getCurrentCode())
30    def editClick(self):
31        self.__form.editClick()
32        self.__table.update()
33    def delClick(self):
34        self.__form.delClick()
35        self.__table.update()
36    def tableClick(self):
37        self.__form.setCurrentCode(self.__table.getCurrentCode())
38    def setConnect(self):
39        self.__newButton.clicked.connect(self.newClick)
40        self.__editButton.clicked.connect(self.editClick)
41        self.__delButton.clicked.connect(self.delClick)
42        self.__table.currentCellChanged.connect(self.tableClick)
43    def update(self):

```

```
44 self.__table.update()
```

4.1.6 Таблицы для сущностей

Таблицы наследуются от `dbTableWidget` и переопределяют `setData()` для заполнения строк данными из соответствующих списков.

Модуль `departmentstable.py`

Листинг 4.6: Модуль `departmentstable.py` — таблица отделов

```
1 from PyQt5.QtWidgets import QTableWidgetItem
2 from dbtablewidget import dbTableWidget
3
4 class departmentsTable(dbTableWidget):
5     def __init__(self, library, parent=None):
6         dbTableWidget.__init__(self, library=library, parent=parent, header=[u'
7             название', u'количество сотрудников'])
8     def setData(self):
9         self.setRowCount(len(self.getLibrary().getDepartmentCodes()))
10        r=0
11        for d in self.getLibrary().getDepartmentList():
12            self.setItem(r,0,QTableWidgetItem(d.getName()))
13            self.setItem(r,1,QTableWidgetItem(str(d.getEmployees())))
14            self.appendRowCode(r,d.getCode())
15            r+=1
```

Модуль `expensetypestable.py`

Листинг 4.7: Модуль `expensetypestable.py` — таблица видов расходов

```
1 from PyQt5.QtWidgets import QTableWidgetItem
2 from dbtablewidget import dbTableWidget
3
4 class expensetypesTable(dbTableWidget):
5     def __init__(self, library, parent=None):
6         dbTableWidget.__init__(self, library=library, parent=parent, header=[u'
7             название', u'описание', u'предельная норма'])
8     def setData(self):
9         self.setRowCount(len(self.getLibrary().getExpenseTypeCodes()))
10        r=0
11        for e in self.getLibrary().getExpenseTypeList():
12            self.setItem(r,0,QTableWidgetItem(e.getName()))
13            self.setItem(r,1,QTableWidgetItem(e.getDescription()))
14            self.setItem(r,2,QTableWidgetItem(str(e.getLimit())))
15            self.appendRowCode(r,e.getCode())
16            r+=1
```

Модуль `expensestable.py`

Листинг 4.8: Модуль `expensestable.py` — таблица расходов

```
1 from PyQt5.QtWidgets import QTableWidgetItem
2 from dbtablewidget import dbTableWidget
3
4 class expensesTable(dbTableWidget):
5     def __init__(self, library, parent=None):
6         dbTableWidget.__init__(self, library, parent=parent, header=[u'
7             вид расхода', u'отдел', u'сумма', u'дата'])
8     def setData(self):
9         self.setRowCount(len(self.getLibrary().getExpenseCodes()))
10        r=0
11        for e in self.getLibrary().getExpenseList():
12            self.setItem(r, 0, QTableWidgetItem(e.getExpenseTypeName()))
13            self.setItem(r, 1, QTableWidgetItem(e.getDepartmentName()))
14            self.setItem(r, 2, QTableWidgetItem(str(e.getAmount())))
15            self.setItem(r, 3, QTableWidgetItem(e.getDate()))
16            self.appendRowCode(r, e.getCode())
17            r+=1
```

4.1.7 Комбо-боксы для выбора связанных объектов

Для формы редактирования расхода созданы два специализированных выпадающих списка, которые заполняются данными из соответствующих списков и автоматически выбирают текущее значение при обновлении.

Модуль `departmentcombo.py`

Листинг 4.9: Модуль `departmentcombo.py` — комбо-бокс выбора отдела

```
1 from dbcombobox import dbComboBox
2
3 class departmentCombo(dbComboBox):
4     def update(self):
5         self.clear()
6         for d in self.getLibrary().getDepartmentList():
7             self.addItem(d.getCode(), d.getName())
8         if self.getCurrentRec() in self.getLibrary().getExpenseCodes():
9             if self.getLibrary().getExpense(self.getCurrentRec()).
10                getDepartment():
11                self.setCurrentCode(self.getLibrary().getExpense(self.
12                    getCurrentRec()).getDepartment().getCode())
```

Модуль `expensetypecombo.py`

Листинг 4.10: Модуль `expensetypecombo.py` — комбо-бокс выбора вида расхода

```
1 from dbcombobox import dbComboBox
2
3 class expensetypeCombo(dbComboBox):
4     def update(self):
5         self.clear()
6         for e in self.getLibrary().getExpenseTypeList():
```

```

7         self.addItem(e.getCode(), e.getName())
8     if self.getCurrentRec() in self.getLibrary().getExpenseCodes():
9         if self.getLibrary().getExpense(self.getCurrentRec()).
            getExpenseType():
10             self.setCurrentCode(self.getLibrary().getExpense(self.
                getCurrentRec()).getExpenseType().getCode())

```

4.1.8 Формы редактирования

Формы наследуются от `editForm` и добавляют поля ввода, соответствующие атрибутам сущностей.

Модуль `departmenteditform.py`

Листинг 4.11: Модуль `departmenteditform.py` — форма редактирования отдела

```

1 from PyQt5.QtWidgets import QLineEdit, QSpinBox
2 from editform import editForm
3
4 class departmentEditForm(editForm):
5     def __init__(self, parent=None, library=None):
6         editForm.__init__(self, library=library, parent=parent)
7         self.__nameEdit = QLineEdit()
8         self.__employeesSpin = QSpinBox()
9         self.__employeesSpin.setRange(0, 100000)
10        self.addLabel(u'название', 0, 0)
11        self.addNewWidget(self.__nameEdit, 0, 1)
12        self.addLabel(u'количество сотрудников', 1, 0)
13        self.addNewWidget(self.__employeesSpin, 1, 1)
14        if self.getLibrary().getDepartmentList():
15            self.setCurrentCode(self.getLibrary().getDepartmentList()[0].
                getCode())
16
17        def update(self):
18            if self.getCurrentCode() in self.getLibrary().getDepartmentCodes():
19                self.__nameEdit.setText(self.getLibrary().getDepartment(self.
                    getCurrentCode()).getName())
20                self.__employeesSpin.setValue(self.getLibrary().getDepartment(
                    self.getCurrentCode()).getEmployees())
21
22        def editClick(self):
23            self.getLibrary().getDepartment(self.getCurrentCode()).setName(self.
                __nameEdit.text())
24            self.getLibrary().getDepartment(self.getCurrentCode()).setEmployees(
                self.__employeesSpin.value())
25
26        def newClick(self):
27            d = self.getLibrary().newDepartment("", 0)
28            self.setCurrentCode(d.getCode())
29
30        def delClick(self):
31            self.getLibrary().removeDepartment(self.getCurrentCode())

```

Модуль `expensetypeeditform.py`

Листинг 4.12: Модуль `expensetypeeditform.py` — форма редактирования вида расхода

```
1 from PyQt5.QtWidgets import QLineEdit, QDoubleSpinBox
2 from editform import editForm
3
4 class expensetypeEditForm(editForm):
5     def __init__(self, parent=None, library=None):
6         editForm.__init__(self, library=library, parent=parent)
7         self.__nameEdit = QLineEdit()
8         self.__descriptionEdit = QLineEdit()
9         self.__limitSpin = QDoubleSpinBox()
10        self.__limitSpin.setRange(0, 100000000)
11        self.addLabel(u'название', 0, 0)
12        self.addNewWidget(self.__nameEdit, 0, 1)
13        self.addLabel(u'описание', 1, 0)
14        self.addNewWidget(self.__descriptionEdit, 1, 1)
15        self.addLabel(u'предельная норма', 2, 0)
16        self.addNewWidget(self.__limitSpin, 2, 1)
17        if self.getLibrary().getExpenseTypeList():
18            self.setCurrentCode(self.getLibrary().getExpenseTypeList()[0].
19                               getCode())
20        def update(self):
21            if self.getCurrentCode() in self.getLibrary().getExpenseTypeCodes():
22                self.__nameEdit.setText(self.getLibrary().getExpenseType(self.
23                                   getCurrentCode()).getName())
24                self.__descriptionEdit.setText(self.getLibrary().getExpenseType(
25                                   self.getCurrentCode()).getDescription())
26                self.__limitSpin.setValue(self.getLibrary().getExpenseType(self.
27                                   getCurrentCode()).getLimit())
28        def editClick(self):
29            self.getLibrary().getExpenseType(self.getCurrentCode()).setName(self
30            .__nameEdit.text())
31            self.getLibrary().getExpenseType(self.getCurrentCode()).
32            setDescription(self.__descriptionEdit.text())
33            self.getLibrary().getExpenseType(self.getCurrentCode()).setLimit(
34            self.__limitSpin.value())
35        def newClick(self):
36            e = self.getLibrary().newExpenseType("", "", 0)
37            self.setCurrentCode(e.getCode())
38        def delClick(self):
39            self.getLibrary().removeExpenseType(self.getCurrentCode())
```

Модуль `expenseeditform.py`

Форма редактирования расхода содержит два комбо-бокса (`expensetypeCombo` и `departmentCombo`) для выбора вида расхода и отдела, поле для суммы (`QDoubleSpinBox`) и поле для даты (`QDateEdit` с календарём).

Листинг 4.13: Модуль `expenseeditform.py` — форма редактирования расхода

```
1 from PyQt5.QtWidgets import QLineEdit, QDoubleSpinBox, QDateEdit
2 from PyQt5.QtCore import QDate
3 from editform import editForm
4 from departmentcombo import departmentCombo
5 from expensetypecombo import expensetypeCombo
6
7 class expenseEditForm(editForm):
8     def __init__(self, parent=None, library=None):
```



```

9      editForm.__init__(self, library=library, parent=parent)
10     self.__expensetypeCombo=expensetypeCombo(library=library)
11     self.__departmentCombo=departmentCombo(library=library)
12     self.__amountSpin=QDoubleSpinBox()
13     self.__amountSpin.setRange(0,10000000)
14     self.__dateEdit=QDateEdit()
15     self.__dateEdit.setCalendarPopup(True)
16     self.__dateEdit.setDate(QDate.currentDate())
17     self.addLabel(u'вид расхода',0,0)
18     self.addNewWidget(self.__expensetypeCombo,0,1)
19     self.addLabel(u'отдел',1,0)
20     self.addNewWidget(self.__departmentCombo,1,1)
21     self.addLabel(u'сумма',2,0)
22     self.addNewWidget(self.__amountSpin,2,1)
23     self.addLabel(u'дата',3,0)
24     self.addNewWidget(self.__dateEdit,3,1)
25     if self.getLibrary().getExpenseList():
26         self.setCurrentCode(self.getLibrary().getExpenseList()[0].
            getCode())
27     def update(self):
28         if self.getCurrentCode() in self.getLibrary().getExpenseCodes():
29             self.__expensetypeCombo.setCurrentRec(self.getCurrentCode())
30             self.__departmentCombo.setCurrentRec(self.getCurrentCode())
31             self.__amountSpin.setValue(self.getLibrary().getExpense(self.
                getCurrentCode()).getAmount())
32             if self.getLibrary().getExpense(self.getCurrentCode()).getDate()
33                 :
34                 self.__dateEdit.setDate(QDate.fromString(self.getLibrary().
                    getExpense(self.getCurrentCode()).getDate(),'yyyy-MM-dd'))
35     def editClick(self):
36         self.getLibrary().getExpense(self.getCurrentCode()).setExpenseType(
            self.getLibrary().getExpenseType(self.__expensetypeCombo.
                getCurrentCode()))
37         self.getLibrary().getExpense(self.getCurrentCode()).setDepartment(
            self.getLibrary().getDepartment(self.__departmentCombo.
                getCurrentCode()))
38         self.getLibrary().getExpense(self.getCurrentCode()).setAmount(self.
            __amountSpin.value())
39         self.getLibrary().getExpense(self.getCurrentCode()).setDate(self.
            __dateEdit.date().toString('yyyy-MM-dd'))
40     def newClick(self):
41         e=self.getLibrary().newExpense("",None,None,0,'')
42         self.setCurrentCode(e.getCode())
43     def delClick(self):
44         self.getLibrary().removeExpense(self.getCurrentCode())

```

4.1.9 Страницы

Каждая страница объединяет соответствующую таблицу и форму редактирования, а также устанавливает соединения между ними.

Модуль departmentpage.py

Листинг 4.14: Модуль departmentpage.py — страница отделов

```

1 from page import page
2 from departmentstable import departmentsTable
3 from departmenteditform import departmentEditForm
4
5 class departmentPage(page):
6     def __init__(self, library, parent=None):
7         page.__init__(self, library=library, parent=parent)
8         self.setTable(departmentsTable(library=library, parent=parent))
9         self.setForm(departmentEditForm(library=library, parent=parent))
10        self.setConnect()

```

Модуль expensetypepage.py

Листинг 4.15: Модуль expensetypepage.py — страница видов расходов

```

1 from page import page
2 from expensetypetable import expensetypesTable
3 from expensetypeeditform import expensetypeEditForm
4
5 class expensetypePage(page):
6     def __init__(self, library, parent=None):
7         page.__init__(self, library=library, parent=parent)
8         self.setTable(expensetypesTable(library=library, parent=parent))
9         self.setForm(expensetypeEditForm(library=library, parent=parent))
10        self.setConnect()

```

Модуль expensepage.py

Листинг 4.16: Модуль expensepage.py — страница расходов

```

1 from page import page
2 from expensetable import expensesTable
3 from expenseeditform import expenseEditForm
4
5 class expensePage(page):
6     def __init__(self, library, parent=None):
7         page.__init__(self, library=library, parent=parent)
8         self.setTable(expensesTable(library=library, parent=parent))
9         self.setForm(expenseEditForm(library=library, parent=parent))
10        self.setConnect()

```

4.1.10 Модуль tabwidget.py

Виджет вкладок, объединяющий три страницы (отделы, виды расходов, расходы). При переключении вкладок обновляет все страницы.

Листинг 4.17: Модуль tabwidget.py — виджет вкладок

```

1 from PyQt5.QtWidgets import QTabWidget
2 from departmentpage import departmentPage
3 from expensetypepage import expensetypePage
4 from expensepage import expensePage
5
6 class tabWidget(QTabWidget):

```

```

7     def __init__(self, accounting, parent=None):
8         QTabWidget.__init__(self, parent)
9         self.__departmentPage=departmentPage(library=accounting)
10        self.addTab(self.__departmentPage, u"отделы")
11        self.__expensetypePage=expensetypePage(library=accounting)
12        self.addTab(self.__expensetypePage, u"виды расходов")
13        self.__expensePage=expensePage(library=accounting)
14        self.addTab(self.__expensePage, u"расходы")
15        self.currentChanged.connect(self.update)
16    def update(self):
17        self.__departmentPage.update()
18        self.__expensetypePage.update()
19        self.__expensePage.update()

```

4.1.11 Модуль mainWindow.py

Главное окно приложения. Содержит меню File с действиями: New (очистка данных), Open XML (загрузка из XML-файла), Save XML (сохранение в XML-файл). При открытии файла данные загружаются через dataxml, после чего все виджеты обновляются.

Листинг 4.18: Модуль mainWindow.py — главное окно приложения

```

1  from PyQt5.QtWidgets import QMainWindow, QAction, QFileDialog
2  from PyQt5.QtGui import QIcon
3  import sys, os
4  from dataxml import dataxml
5  from accounting import accounting
6  from tabwidget import tabWidget
7
8  class mainWindow(QMainWindow):
9      def __init__(self):
10         QMainWindow.__init__(self)
11         self.setWindowTitle(u'Учет внутриофисных расходов')
12         self.setGeometry(100, 100, 800, 600)
13         self.__accounting=accounting()
14         self.dataxml=dataxml(self.__accounting)
15         self.tabWidget=tabWidget(self.__accounting, self)
16         self.setCentralWidget(self.tabWidget)
17         self.tabWidget.update()
18
19         self.new=QAction(QIcon(), 'New', self)
20         self.new.setStatusTip('New database')
21         self.new.triggered.connect(self.newAction)
22
23         self.openxml=QAction(QIcon(), 'Open XML', self)
24         self.openxml.setStatusTip('Open data from XML')
25         self.openxml.triggered.connect(self.openXMLAction)
26
27         self.savexml=QAction(QIcon(), 'Save XML', self)
28         self.savexml.setStatusTip('Save data to XML')
29         self.savexml.triggered.connect(self.saveXMLAction)
30
31         self.menubar=self.menuBar()
32         self.menufile=self.menubar.addMenu('&File')
33         self.menufile.addAction(self.new)
34         self.menufile.addSeparator()

```

```

35         self.menufile.addAction(self.openxml)
36         self.menufile.addSeparator()
37         self.menufile.addAction(self.savexml)
38         self.statusBar()
39     def newAction(self):
40         self.__accounting.clear()
41         self.tabWidget.update()
42     def openXMLAction(self):
43         filename=QFileDialog.getOpenFileName(self,u'Открыть XML',os.getcwd()
44         ,u"*.xml")[0]
45         if filename:
46             self.__accounting.clear()
47             self.dataxml.readFile(filename=filename)
48             self.tabWidget.update()
49     def saveXMLAction(self):
50         filename=QFileDialog.getSaveFileName(self,u'Сохранить XML',os.getcwd
51         (),u"*.xml")[0]
52         if filename:self.dataxml.writeFile(filename=filename)

```

4.1.12 Точка входа: main.pyw

Листинг 4.19: Модуль main.pyw — точка входа в приложение

```

1 from PyQt5.QtWidgets import QApplication
2 from mainwindow import mainWindow
3 import sys
4
5 app = QApplication(sys.argv)
6 mw=mainWindow()
7 mw.show()
8 sys.exit(app.exec_())

```

4.2 UML-диаграмма классов GUI

На рисунке 4.1 представлена диаграмма классов графического интерфейса, отражающая иерархию наследования виджетов и их отношения с классами данных.



При запуске приложения (`python main.pyw`) открывается главное окно с тремя вкладками. Пользователь может создать новую базу данных, загрузить данные из XML-файла или работать с существующими данными.

4.3.1 Вкладка «Отделы»

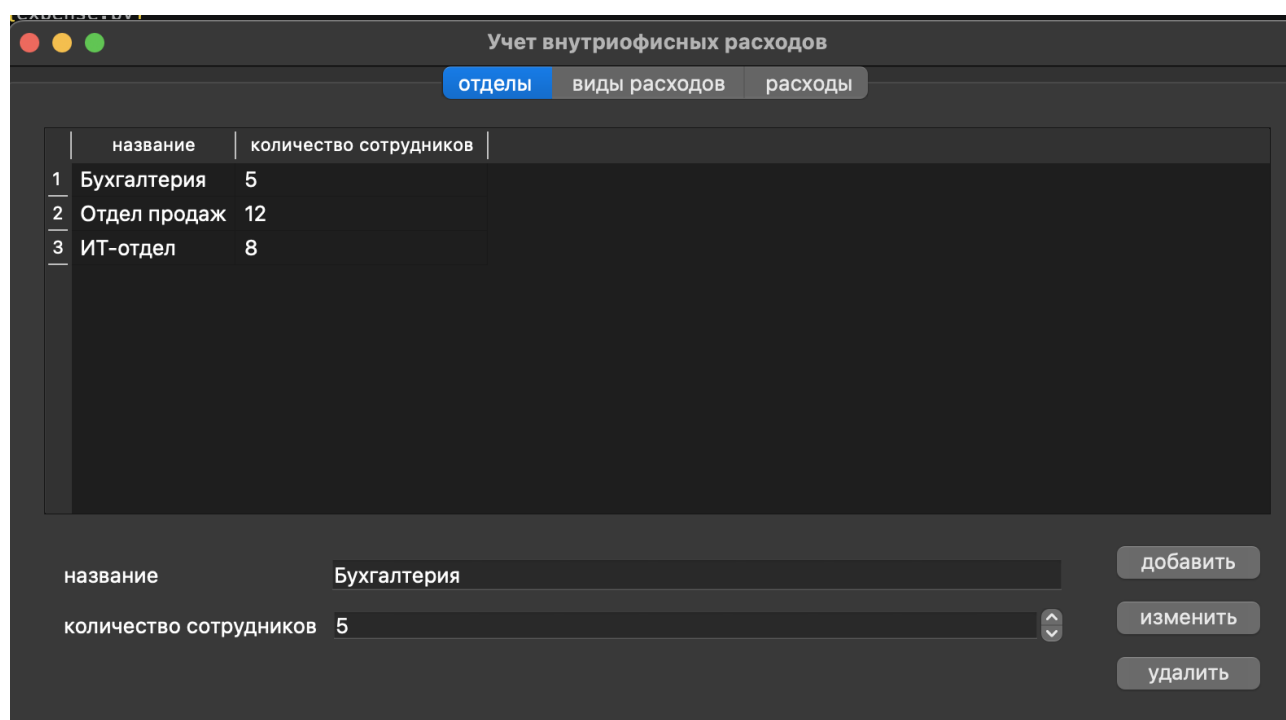


Рис. 4.2: Вкладка «Отделы» с таблицей и формой редактирования

На рисунке 4.2 показана вкладка «Отделы». В верхней части расположена таблица со списком отделов (название, количество сотрудников). В нижней части — форма редактирования, позволяющая добавлять новые отделы, изменять существующие и удалять их.

4.3.2 Вкладка «Виды расходов»

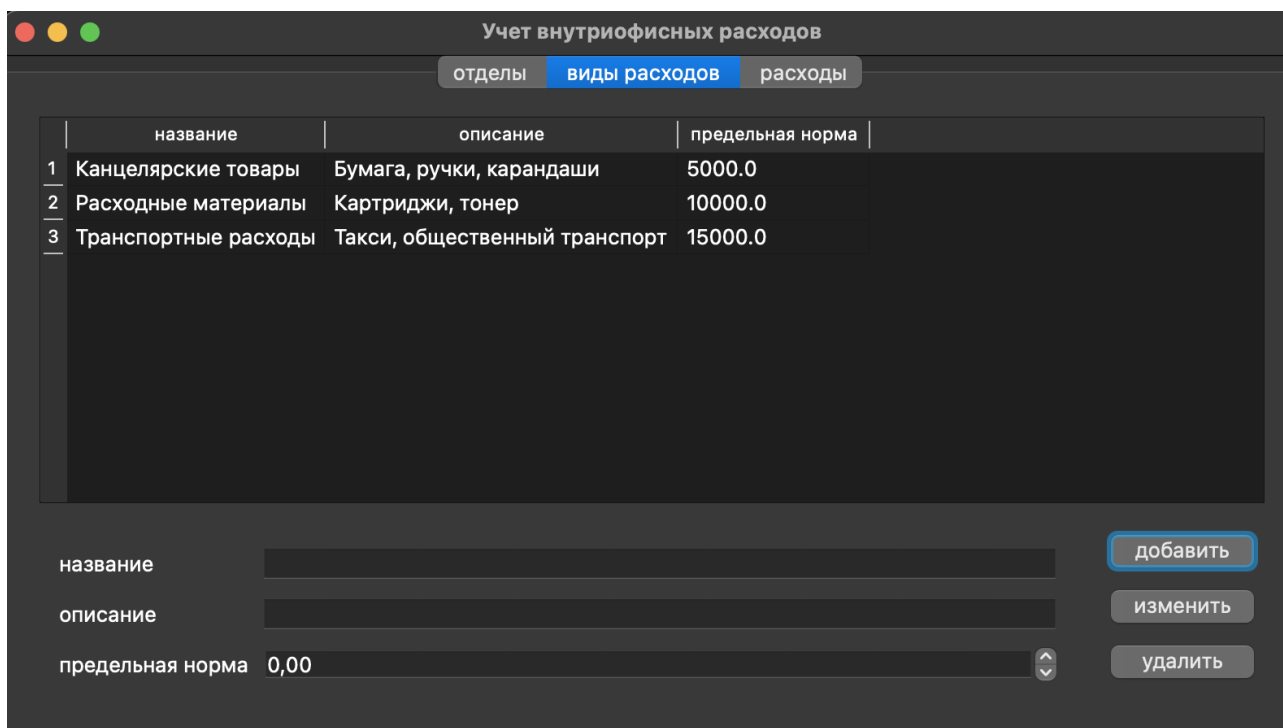


Рис. 4.3: Вкладка «Виды расходов» с таблицей и формой редактирования

На рисунке 4.3 показана вкладка «Виды расходов». Форма редактирования содержит поля: название, описание и предельная норма (с использованием `QDoubleSpinBox`).

4.3.3 Вкладка «Расходы»

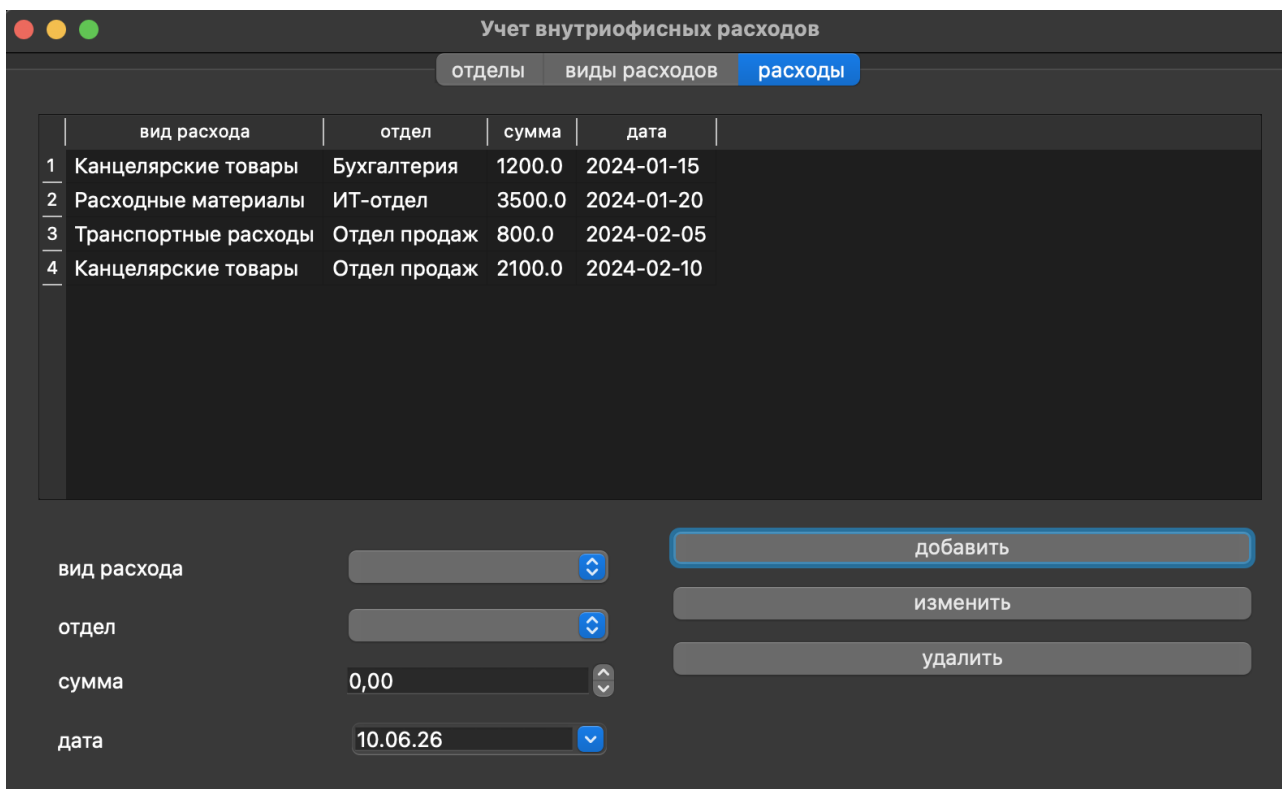


Рис. 4.4: Вкладка «Расходы» с таблицей и формой редактирования

На рисунке 4.4 показана вкладка «Расходы». Форма редактирования содержит выпадающие списки для выбора вида расхода и отдела, поле ввода суммы и календарь для выбора даты.

4.3.4 Загрузка данных из XML

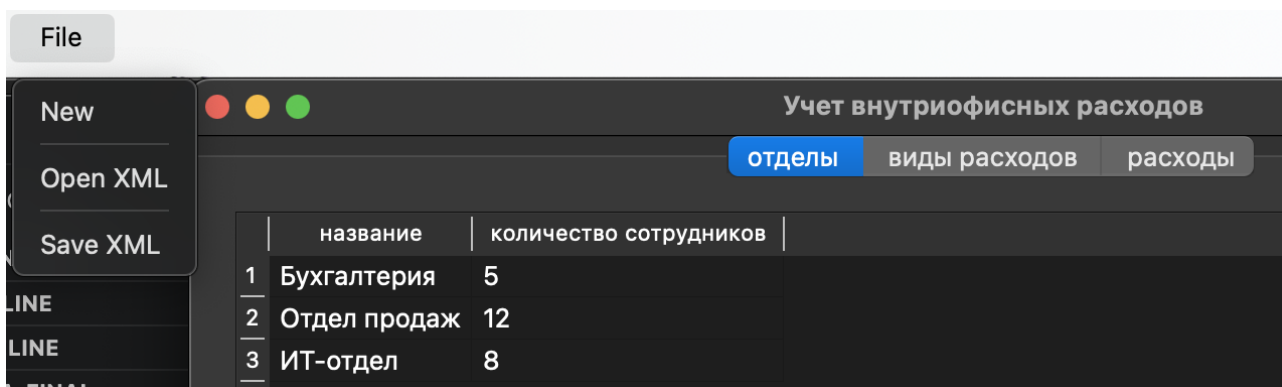


Рис. 4.5: Загрузка данных из XML-файла

На рисунке 4.5 показан результат загрузки данных из файла `data.xml` через меню `File` → `Open XML`. После загрузки все три вкладки заполняются

данными, и связи между расходами, отделами и видами расходов восстанавливаются корректно.

4.3.5 Сохранение данных в XML

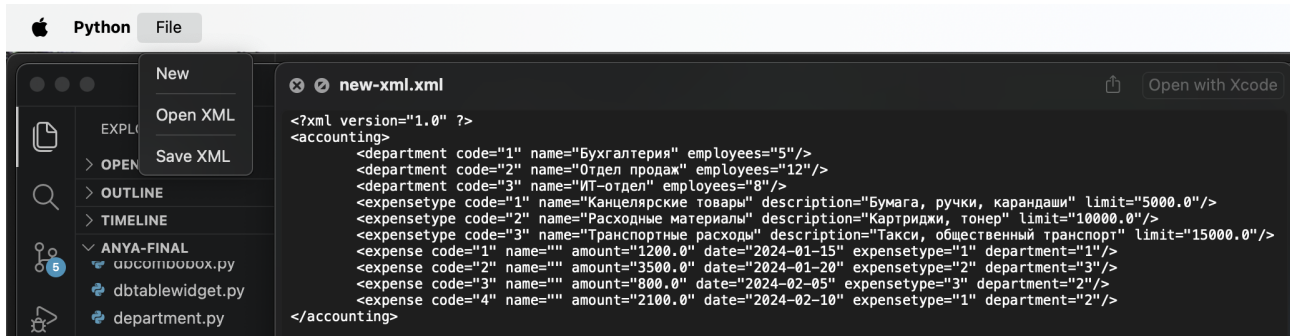


Рис. 4.6: Сохранение данных в XML-файл

На рисунке 4.6 показан процесс сохранения данных. При выборе File → Save XML открывается диалог сохранения файла, после чего данные записываются в формат XML с правильным форматированием и кодировкой UTF-8.

Глава 5

Заключение

В результате выполнения курсовой работы разработано приложение на языке Python с графическим интерфейсом PyQt5 для учёта внутриофисных расходов. Приложение реализует:

- Объектно-ориентированную модель предметной области с тремя сущностями (Отдел, Вид расхода, Расход), построенную на принципах наследования и агрегации.
- Систему хранения данных на основе специализированных списков с централизованным управлением через класс `accounting`.
- Полноценный графический интерфейс с таблицами, формами редактирования, выпадающими списками для выбора связанных объектов и календарём для выбора даты.
- Поддержку сохранения и загрузки данных в формате XML с использованием библиотеки `xml.dom.minidom`.
- Архитектуру, полностью соответствующую шаблонам, представленным в учебных примерах `example4` и `example5`.

Приложение готово к использованию в бухгалтерии частной фирмы для отслеживания внутриофисных расходов сотрудников.

Общий объём исходного кода составляет 31 файл (около 800 строк), что демонстрирует масштабируемость выбранной архитектуры и её пригодность для дальнейшего расширения (добавление поддержки JSON, SQLite и других форматов хранения).